

Sentiment Analysis

LSTM Neural Network for Customer Review Classification

Research Question

Is it possible to build a neural network model trained on previous customer review data capable of processing customer review text records into a positive or negative sentiment classification prediction?

Analysis Goal

The goal of this data analysis will be to clean, tokenize, and analyze statistics on the provided vocabulary dataset to allow for modeling of a neural network capable of a binary classification sentiment analysis built using three datasets of customer reviews. This model will be capable of sentiment analysis on future datasets.

Neural Network Type

The neural network will be built using a long-short term model as one layer. LSTM will be implemented for its ability to break down sequences and strength in retaining relevant information while effectively 'forgetting' the irrelevant.

Exploratory Data Analysis

Unusual characters: There did not appear to be any special characters in the dictionary, but line 60 would replace any special characters and is implemented to be certain and ensure this code is applicable to future datasets.

Vocabulary size: The `tokenizer.word_index` function shows 5271 total words.

Word embedding length: The proposed word embedding length will be 128.

Maximum sequence length: The maximum allowed sequence length is going to be set to the maximum string length of any record in the dataframe provided by the `len().max()` function.

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Tokenization

The goal of the tokenization process will be to convert the records containing cleaned text sequences into sequences of representative integers.

Padding

Zeros are used to pad the text sequences by adding a record with the value 0. Each row will be extended by adding as many necessary records before the tokenized data until each row has as many records as the

longest row.

0	0	0	0	0	271	3	10	13	242	2	54	13	75	69
---	---	---	---	---	-----	---	----	----	-----	---	----	----	----	----

Sentiment Categories & Activation

There will be two sentiment categories, positive and negative. The activation function for the dense layer is 'sigmoid'.

Data Preparation Steps

First, the three datasets, amazon, imdb and yelp are imported and have their datapoints split using '/' to distinguish between records. A function of 'x_df[:-1]' is run over each dataset to remove the final row. Records are then split on tabs. The records are then added into a single newly defined dataframe, containing each record from each dataset.

The dataframe is then used to create a dictionary using pandas to_dict function. The counter function is implemented to list and index each vocabulary word in the dataframe.

All indicators of new line are removed. The first column is adjusted to be entirely lowercase. The maximum length for a record in the dataframe is set equal to the observed maximum string length in column 0. A line is implemented to remove all characters that are not the defined a-z and 0-9, essentially removing all special characters and leaving that space empty.

Then the code will break each sentence into individual words and keep them in a set defined as 'words'. The numeric columns are renamed to 'text' and 'sentiment_label'. Text values are extracted from the dataframe. The tokenizer is initiated and implemented on the previously defined text values, converting the text into integers indexed to the vocabulary word they represent.

The dataset is now padded, meaning all rows will have an equal amount of records by setting any records exceeding the observed record amount per row to 0, up to the value of the longest row plus one. The padding is added to the beginning of each row.

Finally, the dataset can be split 90/10 for a train/test split. X train will have a shape of (2700,480), and X test a shape of (300,480). Y test has a shape of (2700,) and Y train a shape of (300,). The validation data will be set to equal (x_test, y_train) which is ((2700,480),(300,)).

Model Summary

```

# Clean Data - replace new line with empty, lowercase row names, max length, repl spc
df[1] = df[1].map(lambda x : x.replace('\n',''))
df[0] = df[0].str.lower()
maxlen = df[0].str.len().max()
df[0] = df[0].str.replace('[^0-9a-zA-Z.,-/ ]', '')
words = {word for line in df[0]
          for word in line.rsplit(", ", 1)[0].split()}
words
vocablen = len(words)

df.rename(columns = {0:'text',1:'sentiment_label'}, inplace = True)
text = df.text.values

#Tokenizer, convert text to sequences of integers
tokenizer = Tokenizer(num_words=vocablen)
tokenizer.fit_on_texts(text)
encoded_docs = tokenizer.texts_to_sequences(text)

# Pad sequences with 0s beginning of rows
padded_sequence = pad_sequences(encoded_docs, maxlen = maxlen+1)
print(padded_sequence)
pdd_sqnc_df = pd.DataFrame(padded_sequence)
pdd_sqnc_df.to_csv('C:/Users/velez/OneDrive/Desktop/School/D213/Task 2/padded.csv')

print(tokenizer.word_index)

```

This sequential model from tensorflow includes three layers: Embedding with 748,800 parameters, LSTM with 45,360, and Dense with 61. There are 794,221 total parameters, all of which are trainable. One goal of this step was to ensure there were no remaining non-trainable parameters, and this was achieved.

Hyperparameters

Activation functions: The LSTM layer will have an activation function of 'Tanh' to run on non-linear, higher complexity functions. The Dense Layer will have an activation function of sigmoid as it is being used for a binary classification model.

Number of nodes per layer: Each layer will have two nodes in addition to the Layer type classification.

Loss function: Model.compile() will have a loss function of binary cross-entropy, as this sentiment analysis will have a binary classification.

Optimizer: Adam is chosen as the optimizer as it will best allow for a variable learning rate in the neural network.

Stopping criteria: The stopping criteria will allow two epochs with no improvement before it stops training.

Evaluation metric: The selected metric is accuracy in the model.compile() function. Validation accuracy and training accuracy will be modeled against one another on a line graph, along with a second line graph corresponding to the models' losses. There is also a confusion matrix plotting predicted sentiment labels against true sentiment labels.

Training Epochs

```
85/85 ----- 11s 135ms/step - accuracy: 0.9981 - loss: 0.0129 -  
val_accuracy: 0.8300 - val_loss: 0.7846  
Traceback (most recent call last):
```

The model.fit function has the maximum number of epochs set to four, with the caveat that if two epochs consecutively show no improvement, no more epochs will be run.

Model Fitness & Overfitting

The final model experiences a training accuracy loss decrease with a validation loss increase, indicative of an overfitting model. This could indicate the model has memorized the training data but is not fully capable of making predictions on other datasets as accurately.

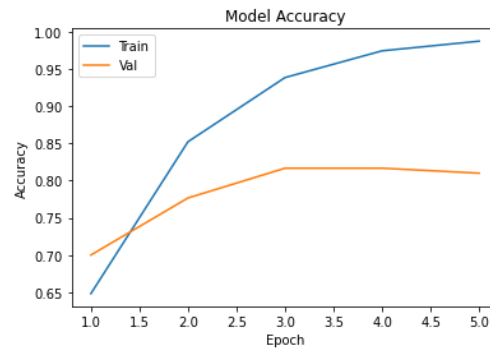
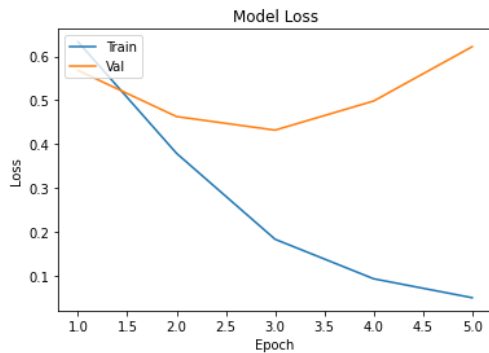
Initially, the model was run using five epochs, with the fifth epoch resulting in validation accuracy decrease. This is addressed by setting the fourth epoch as the final, as this fourth model boasts the highest validation and training accuracy.

Training Visualizations

```
....  
.... print(model.summary())  
Model: "sequential"  


| Layer (type)          | Output Shape     | Param # |
|-----------------------|------------------|---------|
| embedding (Embedding) | (None, 480, 128) | 748,800 |
| lstm (LSTM)           | (None, 60)       | 45,360  |
| dense (Dense)         | (None, 1)        | 61      |

  
Total params: 794,221 (3.03 MB)  
Trainable params: 794,221 (3.03 MB)  
Non-trainable params: 0 (0.00 B)  
None  
In [10]:  
.... early_stopping_monitor = EarlyStopping(patience=2)  
.... history = model.fit(x_train,y_train, epochs=5, batch_size=32,  
validation_data=(x_test,y_test))  
Epoch 1/5  
85/85 ----- 15s 143ms/step - accuracy: 0.6481 - loss: 0.6332 -  
val_accuracy: 0.7000 - val_loss: 0.5680  
Epoch 2/5  
85/85 ----- 12s 141ms/step - accuracy: 0.8522 - loss: 0.3796 -  
val_accuracy: 0.7767 - val_loss: 0.4631  
Epoch 3/5  
85/85 ----- 12s 145ms/step - accuracy: 0.9385 - loss: 0.1842 -  
val_accuracy: 0.8167 - val_loss: 0.4322  
Epoch 4/5  
85/85 ----- 20s 239ms/step - accuracy: 0.9744 - loss: 0.0943 -  
val_accuracy: 0.8167 - val_loss: 0.4985  
Epoch 5/5  
85/85 ----- 22s 253ms/step - accuracy: 0.9874 - loss: 0.0512 -  
val_accuracy: 0.8100 - val_loss: 0.6220
```



Predictive Accuracy

```

.....
.....: model.evaluate(x_test, y_test)
Epoch 1/4
85/85 ----- 13s 148ms/step - accuracy: 0.9889 - loss: 0.0348 -
val_accuracy: 0.8100 - val_loss: 0.6648
Epoch 2/4
85/85 ----- 12s 136ms/step - accuracy: 0.9922 - loss: 0.0305 -
val_accuracy: 0.8200 - val_loss: 0.7164
Epoch 3/4
85/85 ----- 12s 138ms/step - accuracy: 0.9944 - loss: 0.0215 -
val_accuracy: 0.8167 - val_loss: 0.7592
Epoch 4/4
85/85 ----- 11s 135ms/step - accuracy: 0.9981 - loss: 0.0129 -
val_accuracy: 0.8300 - val_loss: 0.7846
Traceback (most recent call last):

```

The training accuracy of the model rises to 99.8% on the final epoch, with a validation accuracy of 83%. This indicates the model is predicting the data it is trained on nearly perfectly, with a significant drop off but still a strong prediction when applied on non-training data. Each epoch consistently decreases the loss of the training data predictions, but loss on the validation data remains mostly consistent.

Saving the Model

```

151
152     model.save("Trained_Model.keras")
153

```

Network Functionality

This neural network is a sentiment analysis model, which takes datasets of customer review text records and converts to counts of positive or negative sentiment prediction.

This model's input layer is embedding, which will train the model on the integers representing the words converted from the tokenizer process. The embedding layer has a trade off in attempting to balance capturing enough dimensions without overfitting by attempting to capture too high a number.

The middle layer, LSTM, will train the model based on relationships between sequences of words. LSTM determines which information the model should forget or output into the next level of processing.

The final Dense layer of the model will use the sigmoid activation function to output a binary classification prediction of that record showing positive or negative sentiment. This is also called the fully connected layer because each neuron in this final layer is a direct result of the previous layers passing on and processing information.

Course of Action

Companies may not be comfortable with 83% accuracy on the validation data. The model's validation accuracy is likely improvable by testing dropout layers and additional hyperparameter tuning, which was implemented, but all tests resulted in less accurate models. When the company is pleased with the validation accuracy, it would be recommended to implement this model on future datasets containing similar review records to quickly provide useful statistics on positive and negative sentiments.